PROJECT BLUEPRINT

Corporate Employee Self-Service (ESS) Portal

Solution Approach, System Architecture
& Technology Stack — Implementation Guide

- End-to-End Architecture
- Module-wise Build Plan
- Database Design
- Security Model
- DevOps & Deployment
- Sprint Roadmap

Prepared as a technical companion to the ESS Portal Requirements Document
Version 1.0 • Engineering Reference Edition

Table of Contents

1. Executive Summary

This document is the engineering companion to the **Corporate Employee Self-Service (ESS) Portal Requirements Document**. Where the requirements document defines *what* the system must do, this guide defines *how* to build it — covering the solution approach, layered system architecture, the complete technology stack with justification for every choice, database design, module-by-module implementation notes, API conventions, security model, project structure, deployment pipeline, non-functional requirements, and a phased delivery roadmap.

The portal is treated as a **modular monolith that can evolve into microservices**: a single Node.js/Express codebase is organized into clearly bounded service modules (Auth, Leave, Attendance, Payroll, Performance, Directory, Training, Helpdesk, Documents) so that any module can later be split into its own deployable service without a rewrite. The frontend is a server-rendered Next.js application for performance and SEO-free internal use, backed by PostgreSQL as the system of record, Redis for caching and queues, and cloud object storage for documents and payslips.

Dimension	Decision at a Glance
Architecture style	Layered, modular-monolith with service boundaries (microservice-ready)
Frontend	Next.js 14 (App Router) + React 18 + Tailwind CSS
Backend	Node.js 20 LTS + Express.js (REST), modular service layer
Database	PostgreSQL 16 (primary), Redis 7 (cache / sessions / queues)
Auth	JWT (access + refresh tokens) + optional SSO via OAuth2 / SAML, MFA via TOTP
Cloud	AWS (or Azure) — ECS/EKS or App Service, S3/Blob Storage, CloudFront/CDN
DevOps	Docker, GitHub Actions CI/CD, IaC via Terraform, centralized logging & monitoring
Delivery model	Agile, 2-week sprints, ~14 sprints (~7 months) to full GA

How to read this document: Sections 2–3 frame the problem and the overall approach. Sections 4–6 cover architecture, tech stack and data design. Section 7 walks through every core module from the requirements document with concrete build notes. Sections 8–12 cover cross-cutting engineering concerns (APIs, security, code structure, DevOps, NFRs). Sections 13–15 cover delivery planning and premium feature notes.

2. Understanding the Problem Statement

The requirements document describes a **multi-tenant-style internal platform** (single organization, multiple departments) that must serve six distinct user roles — Employee, Manager, HR Executive, Payroll Officer, System Administrator, and Department Head — across eleven core modules, plus a set of advanced corporate features (recruitment, onboarding, exit management, travel, expense, visitor and asset management) and a set of recommended premium features (AI assistant, recognition system, digital IDs, internal social feed, wellness, career roadmaps).

2.1 Restating the Core Problem

Strip away the feature list and the underlying engineering problem is: **build a secure, role-aware system of record for the full employee lifecycle** — covering identity, time, money, performance and communication — that different roles can read and write within strict permission boundaries, with audit trails, while staying responsive enough for daily HR operations and payroll-grade financial accuracy.

2.2 Key Engineering Challenges Implied by the Requirements

- **Role-based data visibility:** the same data (e.g. salary, leave balance) must render differently — or not at all — depending on whether the viewer is the employee, their manager, HR, or payroll.
- **Workflow / approval chains:** leave applications, expense claims, and exit requests all need multi-step approval state machines, not simple CRUD.
- **Financial correctness:** payroll and payslips require precise decimal arithmetic, immutable historical records (a payslip must never silently change after issue), and PDF generation at scale.
- **Integration surface:** biometric devices for attendance, payment gateways for reimbursements, email/SMS for notifications, and potentially SSO with an existing corporate identity provider.
- **Compliance and auditability:** GDPR-style data handling, encrypted storage, and audit logs are explicit requirements, not afterthoughts.
- **Scale of relationships:** the data model is dense — almost every module (attendance, leave, payroll, performance, tasks, training, tickets, assets) hangs off a single Employee entity, so the Employees table and its access patterns are the architectural center of gravity.

2.3 Design Principles Adopted to Solve It

1. **Single source of truth per domain** — one canonical table/service owns each type of data; every other module references it by ID rather than duplicating it.
2. **Role-based access control (RBAC) enforced at the API layer**, not just hidden in the UI — every endpoint declares which roles may call it and which fields are returned per role.
3. **Append-only / immutable financial records** — payroll runs and payslips are generated once and stored as immutable snapshots; corrections create new records, never silent overwrites.
4. **Stateless backend services** behind JWT auth, so the API layer can scale horizontally without sticky sessions.
5. **Asynchronous processing for heavy work** — payslip PDF generation, bulk notifications, and biometric sync run through background job queues (Redis-backed) instead of blocking user requests.
6. **Progressive delivery** — core modules ship first (Sections 6–10 of the requirements doc), advanced and premium features follow once the foundation is stable (see Section 13, Roadmap).

3. Solution Approach & Engineering Methodology

3.1 Overall Strategy

The solution is built bottom-up in four layers, each independently testable: **(1) data layer** — schema design and migrations; **(2) service layer** — business logic and rules per module; **(3) API layer** — REST endpoints with RBAC and validation; **(4) presentation layer** — Next.js pages and components consuming the API. This separation means the same API can later serve a future mobile app or third-party integration without change.

3.2 Build Order (Why This Sequence)

1. **Foundation first:** Auth, RBAC, Employee & Department schema — every other module depends on "who is this user and what can they see."
2. **Time & people next:** Attendance and Leave Management — these generate the data Payroll needs.
3. **Money third:** Payroll & Payslips — built once attendance/leave data exists to compute against.
4. **Engagement layer:** Performance, Tasks, Directory, Announcements — lower-risk, higher-iteration modules suited to rapid feedback cycles.
5. **Support layer:** Training, Document Repository, Helpdesk — largely independent, can be built in parallel by a second workstream.
6. **Advanced & premium features:** layered on top once the core eleven modules are stable in production.

3.3 Cross-Cutting Concerns Solved Once, Reused Everywhere

Rather than re-solving the same problem in every module, five concerns are built as shared platform services that every module plugs into:

Concern	Shared Solution
Authentication	Central Auth service issuing JWTs; every other service trusts the token, never re-implements login
Authorization	A single RBAC middleware reads role + permissions from the token on every request
Validation	Shared schema-validation layer (Zod/Joi) — request shapes defined once, reused by API and forms
Notifications	One Notification service (email/SMS/in-app) called by every module instead of each module integrating its own
Audit logging	A single audit-log middleware records who changed what, when — independent of module

3.4 Quality & Delivery Practices

- **Agile delivery:** 2-week sprints with a demoable increment every sprint (see Section 13 roadmap).
- **Trunk-based development** with short-lived feature branches and mandatory pull-request review.
- **Automated testing pyramid:** unit tests for business logic, integration tests for API endpoints, a thin layer of end-to-end tests for critical flows (login, apply leave, run payroll).
- **Contract-first APIs:** OpenAPI/Swagger spec written before implementation so frontend and backend teams can work in parallel against a mock server.
- **Infrastructure as Code** from day one, so environments (dev/staging/prod) are reproducible.

4. System Architecture

The system follows a **layered architecture**: clients talk to an edge layer (CDN + API gateway), which routes to a set of modular backend services, which in turn read/write a shared data layer. Every arrow in the diagram below crosses a trust boundary and is therefore where authentication, validation, or encryption is enforced.

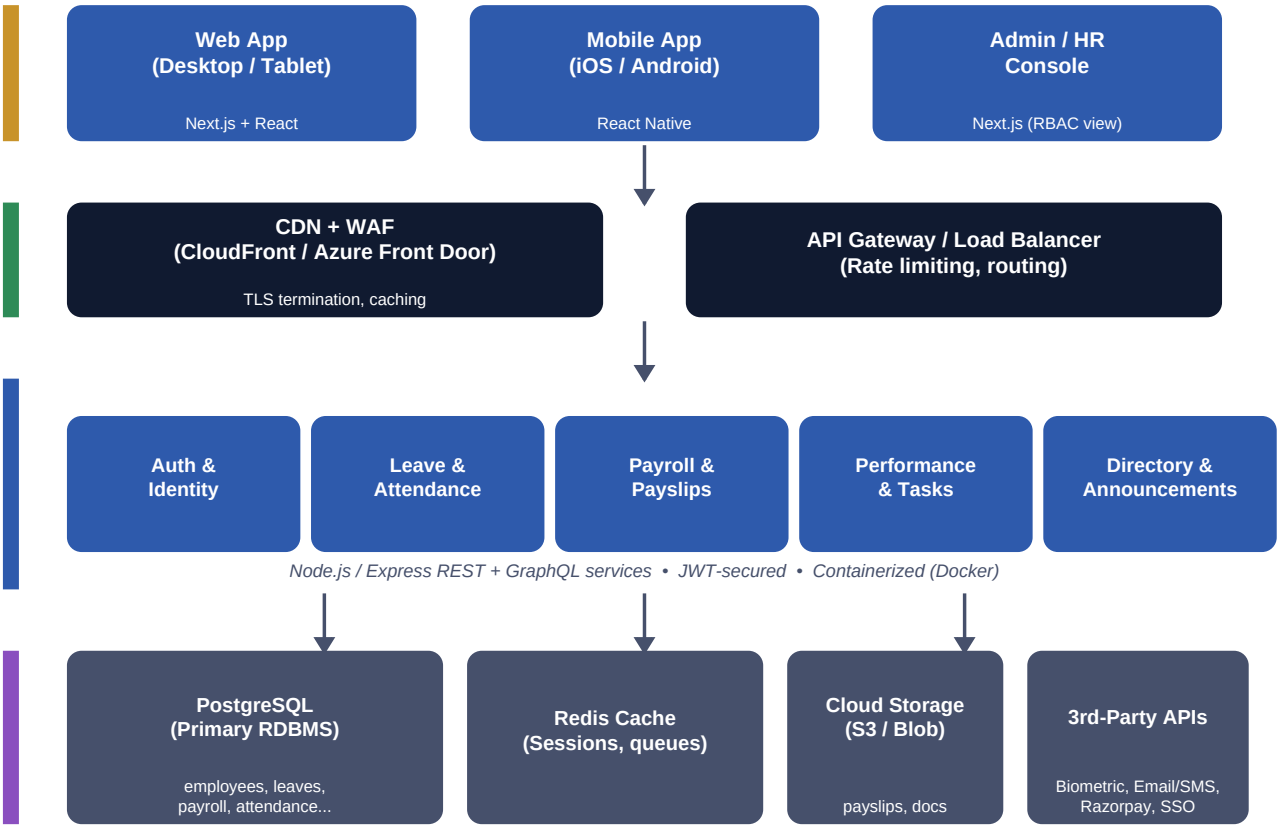

Figure 1 — Layered system architecture: client layer, edge/gateway layer, application service layer, and data/integration layer.

4.1 Client Layer

Three client surfaces share one API: the responsive web app (used by all roles), a future mobile app for on-the-go attendance/leave actions, and a permission-gated admin console for HR/Payroll/Admin roles. All three authenticate the same way and call the same versioned REST API — there is no parallel “admin API.”

4.2 Edge / Gateway Layer

A CDN with a Web Application Firewall (WAF) terminates TLS, caches static assets, and filters common attacks before traffic reaches the application. Behind it, an API Gateway / load balancer handles request routing, rate limiting (protects payroll and auth endpoints from abuse), and health-check-based traffic shifting during deployments.

4.3 Application / Service Layer

Backend logic is organized into five cohesive service groups, each owning its own routes, controllers, and data-access code, but sharing one deployable Node.js/Express process initially (modular monolith). Every service is containerized identically so any of them can be extracted into an independent deployment later with no architectural rework:

- **Auth & Identity** — login, MFA, JWT issuance/refresh, RBAC permission checks, SSO bridge.

- **Leave & Attendance** — leave application/approval workflow, biometric sync, shift & overtime calculation.
- **Payroll & Payslips** — salary computation, tax/deduction rules, payslip PDF generation, payroll reports.
- **Performance & Tasks** — goals, KPIs, reviews, manager feedback, task assignment/tracking.
- **Directory & Announcements** — org chart, employee search, company-wide and team announcements.

Training, Document Repository, and Helpdesk are implemented as two additional internal services (Content & Learning, Support & Ticketing) that follow the same pattern; they are omitted from the diagram only for visual clarity.

4.4 Data & Integration Layer

PostgreSQL is the single system of record for all structured data. Redis serves two purposes: a fast cache for read-heavy data (org chart, announcements, dashboard summaries) and a job queue for asynchronous work (payslip generation, bulk emails, biometric batch sync). Cloud object storage (S3 or Azure Blob) holds binary assets — payslip PDFs, uploaded documents, profile photos, training certificates. Third-party APIs are isolated behind dedicated integration adapters so a provider swap (e.g. changing the SMS vendor) never touches business logic.

5. Technology Stack — Detailed Breakdown

Every choice below follows directly from the requirements document's Section 16 (Technology Stack) and is expanded with the supporting libraries and the reasoning behind each pick.

5.1 Frontend

Layer	Choice	Why
Framework	Next.js 14 (App Router)	Server-side rendering for fast first paint, file-based routing, built-in API routes for BFF needs
UI library	React 18	Component model matches the modular nature of dashboards/widgets across roles
Styling	Tailwind CSS	Utility-first, fast to theme consistently across many screens/roles, small production CSS
Component kit	Headless UI / Radix primitives	Accessible building blocks (modals, dropdowns) without imposing visual style
State / data fetching	React Query (TanStack Query)	Caching, background refetch and loading/error states for API calls, out of the box
Forms	React Hook Form + Zod	Performant forms with schema validation shared with the backend contract
Charts	Recharts / Chart.js	HR analytics dashboards, attendance and leave utilization charts

5.2 Backend

Layer	Choice	Why
Runtime	Node.js 20 LTS	Same language as frontend (TypeScript end-to-end), large ecosystem, good async I/O for an API-heavy system
Framework	Express.js	Minimal, well-understood, easy to organize into modular routers per service group
Language	TypeScript	Type safety across a data model with many interrelated entities reduces runtime bugs
Validation	Zod	Shared validation schemas between frontend and backend
ORM / query layer	Prisma (over PostgreSQL)	Type-safe queries, migrations as code, good fit for a relational, foreign-key-heavy schema
Background jobs	BullMQ (Redis-backed)	Payslip PDF generation, bulk notifications, biometric sync run off the request path
PDF generation	Puppeteer / pdf-lib	Server-side payslip and report PDF rendering

5.3 Database & Storage

Layer	Choice	Why
Primary database	PostgreSQL 16	ACID guarantees for payroll/financial data, mature support for complex relational queries and reporting

Layer	Choice	Why
Cache / queue	Redis 7	Session/token cache, rate-limit counters, BullMQ job queue backend
Object storage	AWS S3 (or Azure Blob)	Payslip PDFs, uploaded documents, profile photos, training certificates
Search (optional, future)	PostgreSQL full-text / OpenSearch	Employee directory and document search at scale

5.4 Authentication, Security & Integrations

Concern	Choice	Why
Auth tokens	JWT (short-lived access + rotating refresh token)	Stateless verification across distributed services
SSO (optional)	OAuth 2.0 / SAML bridge	Integrates with an existing corporate identity provider (Azure AD, Okta, Google Workspace)
MFA	TOTP (e.g. via speakeasy / otplib)	Second factor for sensitive roles (Payroll, Admin) per requirements Section 14
Payments	Razorpay	Reimbursement / expense payouts, as named in the requirements document
Email / SMS	SendGrid / Amazon SES + Twilio	Notification service used by every module uniformly
Biometric integration	Vendor SDK / REST adapter behind an Attendance integration layer	Decouples specific hardware vendor from core attendance logic

5.5 Cloud & DevOps

Concern	Choice	Why
Cloud provider	AWS (primary) — Azure/Vercel as alternatives per requirements	Mature managed services for compute, DB, storage, and CDN
Compute	ECS Fargate or EKS (containers); Vercel for the Next.js frontend if preferred	Scales backend services independently of the frontend
CDN / WAF	CloudFront + AWS WAF	Static asset caching and attack filtering at the edge
CI/CD	GitHub Actions	Lint, test, build, and deploy pipelines triggered per pull request and merge
IaC	Terraform	Reproducible dev/staging/prod environments
Monitoring	CloudWatch / Grafana + Prometheus, Sentry for errors	Operational visibility and alerting

6. Database Design

The schema mirrors the eleven tables named in the requirements document's Section 17, organized as a hub-and-spoke model with **Employees** as the hub. Every spoke table stores an employee_id foreign key, which is what makes role-based queries (“show me everything for this employee” or “show me my team's leave requests”) consistent across modules.

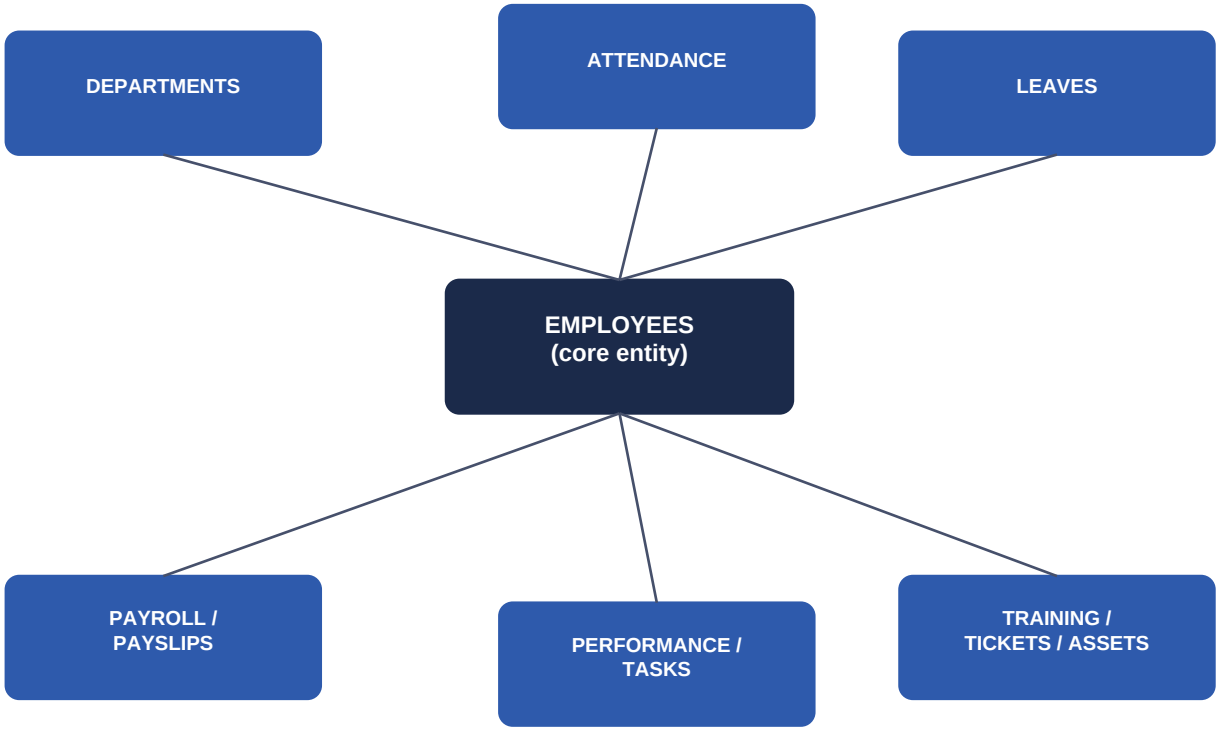

Figure 2 — Simplified entity relationship map: Employees as the central entity, referenced by every other table.

6.1 Core Table Schemas

employees

Field	Type	Notes
id	UUID, PK	Primary identifier referenced by every other table
employee_code	VARCHAR, unique	Human-readable ID badge/login reference
first_name, last_name	VARCHAR	—
email	VARCHAR, unique	Login identifier
password_hash	VARCHAR	bcrypt/argon2 hash; never store plaintext
role	ENUM	employee / manager / hr / payroll / admin / dept_head
department_id	UUID, FK -> departments.id	—

Field	Type	Notes
manager_id	UUID, FK -> employees.id (nullable)	Self-referencing for reporting hierarchy
date_of_joining	DATE	—
status	ENUM	active / on_leave / exited
created_at, updated_at	TIMESTAMP	Audit timestamps

departments

Field	Type	Notes
id	UUID, PK	—
name	VARCHAR	—
head_employee_id	UUID, FK -> employees.id	Department Head role
parent_department_id	UUID, FK (nullable)	Supports nested org structure

attendance

Field	Type	Notes
id	UUID, PK	—
employee_id	UUID, FK -> employees.id	—
date	DATE	—
check_in, check_out	TIMESTAMP	From biometric device or manual entry
work_hours, overtime_hours	DECIMAL	Computed server-side from check_in/out
source	ENUM	biometric / manual / mobile_geo

leaves

Field	Type	Notes
id	UUID, PK	—
employee_id	UUID, FK -> employees.id	—
leave_type	ENUM	casual / sick / earned / unpaid / maternity / paternity
start_date, end_date	DATE	—
status	ENUM	pending / approved / rejected / cancelled
approver_id	UUID, FK -> employees.id	Manager who actions the request

Field	Type	Notes
applied_at, decided_at	TIMESTAMP	Workflow audit trail

payroll / payslips

Field	Type	Notes
id	UUID, PK	—
employee_id	UUID, FK -> employees.id	—
pay_period	VARCHAR (YYYY-MM)	—
gross_salary, net_salary	DECIMAL(12,2)	Stored as fixed-point decimal, never float
deductions_json, allowances_json	JSONB	Itemized breakdown for the payslip PDF
status	ENUM	draft / processed / paid
pdf_url	VARCHAR	Pointer into object storage; payslip is immutable once issued

performance / tasks

Field	Type	Notes
id	UUID, PK	—
employee_id	UUID, FK -> employees.id	—
goal_title, kpi_metric	VARCHAR	—
review_cycle	VARCHAR	e.g. 2026-H1
rating, manager_feedback	DECIMAL / TEXT	—
status	ENUM	open / in_review / closed

announcements / training / tickets / assets

Field	Type	Notes
id	UUID, PK	Each is its own table following the same pattern
employee_id (where applicable)	UUID, FK	Ticket raiser, training enrollee, asset holder
title, body / course_name / subject / asset_tag	VARCHAR/TEXT	Domain-specific payload
status	ENUM	Domain-specific workflow state
created_at, updated_at	TIMESTAMP	—

6.2 Indexing & Integrity Rules

- Foreign keys (employee_id, department_id, manager_id, approver_id) are indexed for fast role-scoped queries.
- Composite index on (employee_id, date) in attendance for daily/range lookups.
- Composite index on (employee_id, pay_period) in payroll, unique constraint to prevent duplicate payroll runs.
- Soft-delete pattern (status = exited) on employees instead of hard deletes, to preserve historical payroll/attendance integrity.
- Row-level audit logging on payroll, leaves, and employees tables specifically, since these are the highest-sensitivity domains.

7. Module-Wise Solution Design

This section walks through each of the eleven core modules from the requirements document and explains how it is solved in practice: the primary user flows, the key API endpoints, and the engineering notes specific to that module.

7.1 Employee Dashboard

User Flows

- On login, the dashboard aggregates data from multiple services into one summary view: profile, attendance %, leave balance, latest payslip amount, announcements, and upcoming holidays.
- Quick actions (apply leave, view payslip, raise ticket) deep-link directly into the relevant module.

Key Endpoints

Method	Endpoint	Purpose
GET	/api/dashboard/summary	Aggregated widget data for the logged-in employee
GET	/api/dashboard/notifications	Unread notification list

Engineering Notes

- Aggregation happens server-side in a dedicated BFF (backend-for-frontend) endpoint to avoid the frontend firing 6+ parallel requests on every page load.
- Heavy widgets (announcements, holiday calendar) are cached in Redis with a short TTL since they change infrequently across all employees.

7.2 Leave Management System

User Flows

- Employee applies for leave, selecting type and date range; system checks balance and team calendar before allowing submission.
- Request routes to the employee's manager for approval; manager approves/rejects with a comment.
- On approval, leave balance is decremented and the leave calendar / attendance record is updated automatically.

Key Endpoints

Method	Endpoint	Purpose
POST	/api/leaves	Submit a new leave application
GET	/api/leaves/balance	Current balances by leave type
PATCH	/api/leaves/{id}/decision	Manager approves or rejects (role-checked)
GET	/api/leaves/calendar	Team/department leave calendar view

Engineering Notes

- Modeled as an explicit state machine (pending → approved/rejected → cancelled) rather than a free-text status field, so invalid transitions are rejected at the API layer.
- Leave balance accrual (e.g. monthly earned leave credit) runs as a scheduled background job, not computed ad hoc on every read.

7.3 Attendance Management

User Flows

- Daily check-in/check-out captured via biometric device sync, manual entry (with manager override permission), or mobile geo-tagged punch.
- Work hours and overtime are computed nightly and surfaced on the dashboard and in attendance reports.

Key Endpoints

Method	Endpoint	Purpose
POST	/api/attendance/sync	Bulk ingest from biometric device (service account only)
POST	/api/attendance/manual	Manual entry/correction (manager+ role)
GET	/api/attendance/report	Date-range attendance and overtime report

Engineering Notes

- Biometric integration is isolated behind an adapter interface so the specific vendor SDK can change without touching attendance business logic.
- Shift management (rotating shifts) is modeled as a separate shifts reference table joined against attendance for overtime calculation.

7.4 Payroll & Payslips

User Flows

- Payroll Officer triggers a monthly payroll run; system computes gross/net pay per employee from base salary, attendance, approved leave, and configured deduction/allowance rules.
- Once processed, individual payslip PDFs are generated and made available for employee download; payroll reports are generated for Finance/HR.

Key Endpoints

Method	Endpoint	Purpose
POST	/api/payroll/run	Trigger a payroll run for a pay period (payroll role only)
GET	/api/payslips/{employeeId}	List/download payslips for an employee
GET	/api/payroll/reports	Aggregate payroll reports (payroll/HR role only)

Engineering Notes

- This is the highest-integrity module in the system: monetary fields use fixed-point DECIMAL types, never floating point, and a payroll run is processed inside a database transaction so it is all-or-nothing.
- Payslip PDF generation runs as an async background job (BullMQ) so a 500-employee run does not block the API request.
- Once status = paid, a payslip record is immutable; corrections create a new linked adjustment record, never an in-place edit.

7.5 Employee Directory

User Flows

- Employees search colleagues by name, department, or role; results show contact info and reporting line, scoped to what the viewer's role is permitted to see.
- Org chart view renders the reporting hierarchy using the self-referencing manager_id field.

Key Endpoints

Method	Endpoint	Purpose
GET	/api/directory/search	Search employees by name/department
GET	/api/directory/{id}/org-chart	Reporting hierarchy around an employee

Engineering Notes

- Directory search uses PostgreSQL full-text search initially; can move to OpenSearch if the employee count or query complexity grows.
- Sensitive fields (personal phone, salary) are excluded from directory responses by default field-level RBAC, regardless of viewer role.

7.6 Performance Management

User Flows

- Manager and employee jointly set goals/KPIs for a review cycle; employee tracks progress; manager leaves periodic feedback and a formal rating at cycle close.
- HR can pull cross-team performance and promotion-recommendation reports.

Key Endpoints

Method	Endpoint	Purpose
POST	/api/performance/goals	Create a goal/KPI for a review cycle
PATCH	/api/performance/goals/{id}	Update progress or status
POST	/api/performance/reviews/{employeeId}	Manager submits feedback/rating

Engineering Notes

- Review cycles are first-class entities (e.g. 2026-H1) so historical reviews are queryable and comparable across cycles.
- Rating scales are configurable (not hard-coded), since different organizations standardize differently.

7.7 Task Management

User Flows

- Managers assign tasks to direct reports (or themselves); employees update task status; overdue tasks surface on the dashboard.

Key Endpoints

Method	Endpoint	Purpose
POST	/api/tasks	Create/assign a task
PATCH	/api/tasks/{id}/status	Update task status

Method	Endpoint	Purpose
GET	/api/tasks/mine	Tasks assigned to the logged-in employee

Engineering Notes

- Kept intentionally lightweight (not a full project-management tool) — status enum (todo/in_progress/done/blocked), due date, and priority are sufficient per the requirements scope.

7.8 Announcements

User Flows

- HR/Admin publish company-wide or department-scoped announcements; employees see them on the dashboard and in a dedicated feed, with read/unread tracking.

Key Endpoints

Method	Endpoint	Purpose
POST	/api/announcements	Publish an announcement (HR/Admin role)
GET	/api/announcements/feed	Scoped feed for the logged-in employee

Engineering Notes

- Targeting (company-wide vs. specific department) is modeled as an optional department_id filter rather than a separate table per audience type.
- Feed reads are cached per department in Redis and invalidated on new publish.

7.9 Training & Learning Center

User Flows

- Employees browse available courses, enroll, track progress, and download completion certificates; HR manages the course catalog and schedules.

Key Endpoints

Method	Endpoint	Purpose
GET	/api/training/courses	Browse course catalog
POST	/api/training/enrollments	Enroll in a course
GET	/api/training/certificates/{employeeId}	Completion certificates

Engineering Notes

- Course content/files are stored in object storage; the database holds only metadata and progress state, keeping the relational schema light.
- Certificate PDFs are generated the same way as payslips, reusing the shared PDF-generation service.

7.10 Document Repository

User Flows

- Employees and HR upload/retrieve documents (offer letters, ID proofs, policy documents) scoped by visibility rules (personal vs. organization-wide).

Key Endpoints

Method	Endpoint	Purpose
POST	/api/documents	Upload a document (pre-signed URL flow)
GET	/api/documents/{id}	Download/view a document (RBAC + ownership checked)

Engineering Notes

- Uploads use pre-signed S3/Blob URLs so file bytes never pass through the application server, improving both performance and security.
- Every document carries an owner, a visibility scope, and a category, enforced identically to the RBAC pattern used elsewhere.

7.11 Helpdesk & Ticketing

User Flows

- Employees raise HR or IT support tickets; tickets route to the appropriate team queue, escalate if unresolved past SLA, and notify the requester on status changes.

Key Endpoints

Method	Endpoint	Purpose
POST	/api/tickets	Raise a new ticket (category: HR / IT)
PATCH	/api/tickets/{id}/status	Update status / assign / escalate
GET	/api/tickets/mine	Tickets raised by or assigned to the logged-in user

Engineering Notes

- Ticket state machine: open → in_progress → escalated → resolved → closed, with timestamps captured at each transition for SLA reporting.
- Escalation-on-SLA-breach runs as a scheduled background job rather than being checked synchronously on every read.

8. API Design

All clients (web, mobile, admin console) consume one versioned REST API (/api/v1/...), defined contract-first via an OpenAPI specification so frontend and backend teams can build in parallel against a shared mock.

8.1 Conventions

- **Resource-oriented URLs:** nouns, not verbs — /leaves, not /applyLeave.
- **Standard verbs:** GET (read), POST (create), PATCH (partial update), DELETE (soft-delete where applicable).
- **Consistent envelope:** every response returns { data, meta, error }, so pagination and error handling are predictable across all 11 modules.
- **Pagination:** cursor-based for high-volume lists (attendance, audit logs), offset-based for small, bounded lists (departments, course catalog).
- **Idempotency keys** on financial mutating calls (POST /payroll/run) to make retries safe.
- **Versioning:** breaking changes ship as /api/v2 rather than mutating v1 in place.

8.2 Example Request / Response

Apply for leave — POST /api/v1/leaves

```
{
  "leaveType": "earned",
  "startDate": "2026-07-14",
  "endDate": "2026-07-16",
  "reason": "Family event"
}
```

Response — 201 Created

```
{
  "data": { "id": "lv_8821", "status": "pending", "approverId": "emp_0142" },
  "meta": { "requestId": "req_a93f" },
  "error": null
}
```

8.3 Error Format

Errors are machine-parseable and role-safe (no internal stack traces leak to the client): { "error": { "code": "LEAVE_BALANCE_INSUFFICIENT", "message": "..." } }. Standard HTTP status codes are used (400 validation, 401 unauthenticated, 403 unauthorized, 404 not found, 409 conflict, 422 business-rule violation, 500 server error).

9. Security Architecture

Security is implemented as defense-in-depth across network, application, and data layers, directly addressing requirements Section 14 (role-based access control, MFA, encryption, audit logs, session management, GDPR-compliant practices).

9.1 Authentication & Session Management

- Passwords hashed with bcrypt/argon2; never stored or logged in plaintext.
- JWT access tokens are short-lived (≈15 min); refresh tokens are longer-lived, rotated on use, and revocable server-side (stored hashed in Redis) — so a compromised refresh token can be invalidated.
- Multi-factor authentication (TOTP) is mandatory for Payroll Officer, System Administrator, and HR Executive roles, and optional-but-encouraged for all others.
- Optional enterprise SSO (OAuth2/SAML) bridges into the same JWT-based session model so downstream services don't need separate logic.

9.2 Role-Based Access Control (RBAC)

Every API request carries a role claim in its JWT; a single shared middleware checks the requested resource and action against a permission matrix before the controller ever runs. Field-level restrictions (e.g. salary visible to Payroll/HR/self only) are enforced in the serialization layer, not just hidden in the UI — meaning a direct API call cannot bypass the restriction either.

Role	Representative Access
Employee	Own profile, attendance, leave, payslips, tasks, tickets — read/write own data only
Manager	All of the above + approve team leave, assign tasks, submit performance reviews for reports
HR Executive	Org-wide employee records, announcements, training, recruitment/onboarding workflows
Payroll Officer	Payroll runs, payslips, compensation data — most sensitive financial scope
Department Head	Department-scoped analytics, approvals, and directory management
System Administrator	User/role management, system configuration, audit log access

9.3 Data Protection

- Encryption in transit: TLS 1.2+ everywhere (enforced at the CDN/WAF and between internal services).
- Encryption at rest: database and object storage encryption enabled at the cloud-provider level (e.g. RDS/S3 KMS-managed keys).
- PII minimization: sensitive fields (national ID, bank account) are stored in a restricted sub-table with stricter access logging than the rest of the employee record.
- GDPR-aligned practices: data export and "right to be forgotten" handled via a documented offboarding process that anonymizes rather than deletes records with payroll/legal retention requirements.

9.4 Audit Logging & Monitoring

A shared audit-log middleware records actor, action, target resource, and timestamp for every mutating request on sensitive modules (Payroll, Leave approvals, Employee records, Admin actions). Logs are append-only and shipped to a separate log store so they cannot be altered by the same credentials that generated them. Anomaly

alerts (e.g. unusual payroll access patterns) integrate with the monitoring stack described in Section 11.

10. Project & Folder Structure

A monorepo keeps the Next.js frontend and the modular Express backend versioned together, sharing TypeScript types and validation schemas via an internal shared package.

```
ess-portal/
|-- apps/
|  |-- web/        # Next.js frontend
|  |  |-- app/       # App Router pages (dashboard, leave, payroll, ...)
|  |  |-- components/
|  |  `-- lib/       # API client, hooks
|  `-- api/        # Express backend
|      |-- src/
|      |  |-- modules/
|      |  |  |-- auth/     # routes, controller, service, validation
|      |  |  |-- leave/
|      |  |  |-- attendance/
|      |  |  |-- payroll/
|      |  |  |-- performance/
|      |  |  `-- ... (one folder per module)
|      |  |-- middleware/  # auth, rbac, validation, audit-log
|      |  |-- jobs/      # BullMQ workers (payslip-pdf, notifications, biometric-sync)
|      |  `-- config/
|      `-- prisma/     # schema.prisma, migrations/
|-- packages/
|  |-- shared-types/  # TypeScript types shared by web + api
|  `-- shared-schemas/ # Zod validation schemas shared by web + api
|-- infra/        # Terraform IaC
`-- .github/workflows/ # CI/CD pipelines
```

Each module folder under modules/ follows the same internal pattern — routes.ts → controller.ts → service.ts → repository.ts → validation.ts — so a developer who understands one module already knows how to navigate any other.

11. DevOps, Deployment & Infrastructure

11.1 Environments

Three persistent environments — **dev**, **staging**, **production** — provisioned identically via Terraform, differing only in scale and data. Staging mirrors production configuration so that performance and integration issues surface before release, not after.

11.2 CI/CD Pipeline

1. **Pull request opened** → GitHub Actions runs lint, type-check, unit tests, and a Docker build.
2. **Merge to main** → integration tests run against an ephemeral database; on success, images are pushed to the container registry and auto-deployed to staging.
3. **Staging verification** → smoke tests and manual QA sign-off.
4. **Production release** → triggered via a tagged release; rolling deployment with health checks, automatic rollback on failed health checks.

11.3 Containerization & Compute

Each backend module-group runs as a container behind the API gateway; the Next.js frontend is deployed either as a containerized service or on a platform like Vercel. Containers are stateless — all session and job state lives in Redis/PostgreSQL — so horizontal scaling is a matter of adding replicas behind the load balancer.

11.4 Observability

- **Logs:** structured JSON logs shipped to a central store (CloudWatch Logs / ELK), correlated by a request ID propagated end-to-end.
- **Metrics:** Prometheus/Grafana dashboards for request latency, error rate, queue depth, and DB connection pool health — the "four golden signals" per service.
- **Error tracking:** Sentry captures unhandled exceptions with stack traces and user/role context (scrubbed of PII).
- **Alerting:** on-call alerts for payroll-run failures, elevated 5xx rates, and failed background jobs.

11.5 Backup & Disaster Recovery

Automated nightly PostgreSQL snapshots with point-in-time recovery enabled; object storage is versioned so an accidental overwrite/deletion of a payslip or document can be recovered. A documented RTO/RPO target (e.g. RTO 4h, RPO 15min for the production database) drives the backup frequency and the disaster-recovery runbook.

12. Non-Functional Requirements

Category	Target
Availability	99.9% uptime for core modules (Auth, Attendance, Leave, Payroll) during business hours
Performance	P95 API response time < 300ms for read endpoints; payroll runs process 1,000 employees in < 5 minutes (async)
Scalability	Stateless services scale horizontally; database read replicas added as directory/reporting load grows
Security	OWASP Top 10 mitigations, quarterly dependency/security audits, annual penetration test
Data integrity	Payroll/leave mutations are transactional; financial figures use fixed-point decimal types
Auditability	Every mutation on Employee, Leave, and Payroll records is logged with actor and timestamp
Accessibility	WCAG 2.1 AA compliance for all employee-facing screens
Browser/device support	Latest two versions of Chrome, Edge, Safari, Firefox; responsive down to mobile widths
Localization-readiness	UI strings externalized for future multi-language support, even if v1 ships English-only

13. Development Roadmap (Sprint Plan)

A 14-sprint (~7 month), 2-week-sprint plan delivers the system in the build order defined in Section 3.2 — foundation first, then time/people data, then payroll, then engagement and support modules, with advanced and premium features layered on afterward.

Sprints	Milestone	Scope
1–2	Foundation	Auth, RBAC, MFA, Employee & Department schema, base UI shell, CI/CD pipeline live
3–4	Attendance & Leave (core)	Attendance capture (manual + biometric adapter), leave application/approval workflow
5–6	Payroll & Payslips	Salary computation engine, payroll run workflow, payslip PDF generation, payroll reports
7	Employee Directory & Dashboard	Org chart, search, dashboard aggregation BFF
8–9	Performance & Tasks	Goals/KPIs, review cycles, manager feedback, task assignment
10	Announcements & Training	Announcement feed, course catalog, enrollment, certificates
11	Document Repository & Helpdesk	Secure document upload/retrieval, ticketing with SLA escalation
12	Reports & Analytics	HR/payroll/attendance/leave analytics dashboards
13	Hardening	Security audit, load testing, accessibility pass, GDPR review
14	Advanced features (start)	Onboarding/exit workflows, expense claims, asset management (foundation for Section 13 of requirements doc)
Post-GA	Premium features	AI HR assistant, recognition system, digital IDs, wellness, career roadmaps (Section 14, this doc)

14. Premium Feature Implementation Notes

The requirements document lists ten recommended premium features. None of them require a change to the core architecture in Section 4 — each is additive and plugs into the existing service layer and data model.

Feature	Implementation Approach
AI-powered HR Assistant	A chat interface backed by an LLM API with retrieval over policy documents and the employee's own data (leave balance, payslips) via existing authenticated endpoints — never given raw database access
Employee Recognition & Rewards	New recognitions table (giver, receiver, message, points); points ledger reuses the existing notification service for alerts
Digital ID Cards	Server-rendered card image/PDF from existing employee profile data; QR code encodes a signed, time-limited verification token
Internal Social Feed	New lightweight posts/comments tables scoped by department/company, reusing the Announcements module's feed and notification patterns
Real-Time Notifications	WebSocket/Server-Sent-Events gateway layered onto the existing Notification service for instant in-app delivery alongside email/SMS
Employee Wellness Programs	New wellness_programs / enrollments tables, modeled identically to the Training module
Career Growth Roadmaps	Extends the Performance module's goals/KPIs with a longer-horizon career_paths table linked to skill data
Skill Assessment Platform	New assessments module following the Training module's enrollment/scoring pattern
Automated Performance Insights	Scheduled analytics job over existing Performance/Attendance data, surfaced on the HR analytics dashboard
Executive Analytics Dashboard	A read-only aggregation layer over existing reporting endpoints (Section 15 of the requirements doc), gated to Department Head/Admin roles

15. Conclusion

The ESS Portal's apparent complexity — eleven core modules, six roles, advanced and premium features — resolves into a manageable engineering problem once it is recognized that almost everything hangs off one well-modeled **Employee** entity, guarded by one consistent **RBAC layer**, and served through one consistent **API contract**. The architecture in this document is deliberately a **microservice-ready modular monolith**: simple enough to build and operate with a small team at launch, but cleanly seamed so that any module — most likely Payroll or Attendance, given their integration surface — can be split into an independently scaled service later without a rewrite.

The recommended path forward is to execute the 14-sprint roadmap in Section 13, instrument the system with the observability stack from Section 11.4 from day one, and treat the security model in Section 9 as non-negotiable given the payroll and personal-data sensitivity of this platform.

End of document — Corporate ESS Portal: Solution Approach, System Architecture & Technology Stack.